SENTINEL: Trustworthy Guardrails for Web-Agent LLM Services

Simarjot Singh Maan* and Quang Bui*
Scientific AI for Development (SAID) Laboratory
**Equal contribution*

Abstract*—Traditional LLM chat APIs, retrieval-augmented assistants, browser automation, and multi-tool orchestrators extends the attack surface beyond user prompts to untrusted web content, encoded payloads, and spoofed agent authorization [6, 19]. Alignment training alone does not provide a deployable trust boundary for these services [12, 20]. Therefore, we present: SENTINEL (Semantic Evaluation, Network Trajectory Integrity, and Node-based Enforcement Logic) - an open inference-time guardrail that wraps any OpenAI-compatible model endpoint with five enforcement nodes (Layers 1–5) under a nine-layer Systemic Alignment Pipeline reference architecture - all done without modifying model weights. The evaluation of this framework uses a 2-track protocol on a curated 226-prompt web-agent benchmark (126 adversarial and 100 benign): reproducible pre-generation ablations and guardrail baselines (Track A) and an end-to-end study on Llama 3.1 8B (Track B) via an OpenAI-compatible local API. On Track A (Layers 1–3), SENTINEL blocks 49.2% of adversarial prompts at 5.0% benign false-positive rate, more than double compared to a keyword baseline (19.0%). Layer 3, the encoding normalization plus context-integrity rules, contributes the largest marginal gain (+35.7 pp over normalization alone), including 86% coverage on indirect tool-injection prompts that model RAG and MCP-style attacks [6]. Track B on Llama 3.1 8B [13] raises end-to-end attack block from 80.2% to 90.5%, cutting true ASR from 19.8% to 9.5%, with 13 incremental layer stops concentrated in prompt leakage and injection.

***Index Terms*—Web intelligence, intelligent agents, web of agents, LLM safety, trust and security, prompt injection, guardrails, reproducible evaluation, trusthworthy AI**

I. Introduction

Web intelligence research increasingly treats large language models as *connected-world services* in which they retrieve documents, invoke tools, and delegate tasks across agent graphs rather than operating as isolated chat interfaces [6, 17, 19]. That connectivity shifts the trust boundary to the API gateway, where user queries, retrieved HTML, and tool JSON are composed before inference. Alignment training improves average refusal behavior but does not, supply the deterministic, low-latency enforcement by itself that web-agent operators need at this boundary [12, 20]. We found out that 3 failure modes motivate inference-time guardrails in this setting. *Untrusted content injection* embeds adversarial instructions inside retrieved pages, emails, or tool outputs so they are interpreted as user intent [6]. *Encoding and obfuscation* wraps harmful requests in Base64, ROT13, or homoglyph forms that lexical filters miss while remaining decodable by the model [5]. *Authorization spoofing* claims prior approval from a supervisor, planner, or tool router to bypass naive input filters [19].

We argue that trustworthy web-agent services require *layered inference-time enforcement* in which it requires deterministic normalization, high-precision rule gates for agentic patterns, semantic classifiers for paraphrased harm, and output scoring before responses reach the users or any downstream tools. **SENTINEL** (**S**emantic **E**valuation, **N**etwork **T**rajectory **I**ntegrity, and **N**ode-based **E**nforcement **L**ogic) implements this proposed design as a model-agnostic wrapper around any OpenAI-compatible endpoint.

Our contributions are fourfold. First, we specify a Systemic Alignment Pipeline reference architecture that maps web-agent threat surfaces to nine enforcement nodes, five of the nodes are implemented in the current prototype of this paper(Section IV). Second, we release a reproducible evaluation artifact of 226 labeled prompts, layer ablations, guardrail baselines, and an explicit labeling protocol—all together with scripts that regenerate all tables and figures (Sections V–VI-D). Third, we empirically show that Layer 3 context-integrity rules dominate the pre-generation defense (+35.7 pp marginal gain), with an 86% block rate on indirect tool-injection prompts (Section VI-C). Fourth, we introduce a defense-credit attribution protocol that separates layer blocks, alignment refusal, and true attack success rate (ASR) in end-to-end evaluation (Section VI-F). Track A supplies primary evidence without LLM variance; Track B reports incremental lift on an already good 80%-refusal Llama 3.1 8B backend.

A. Design principles

SENTINEL differs from single-classifier guardrails in four ways aligned with web-agent deployment.

Compound enforcement. It composes normalization, rules, semantics, and output scoring rather than substituting for alignment training.

Fast-exit routing. It lets Layer 3 regex gates terminate in ∼0.3 ms for 69% of blocks (Table IX), preserving throughput on template jailbreaks and indirect injection.

Web-surface coverage. It includes a dedicated `indirect_tool_injection` rule for poisoned RAG, email, Slack, MCP, and webhook channels (Table II) [6].

Model agnosticism. It attaches to any OpenAI-compatible endpoint without weight access, enabling safety routing across backends.

TABLE I: Guardrail systems for web-agent LLM services

System	Enc.	Ag.	OSS
Llama Guard	Partial	Partial	Yes
NeMo Guardrails	Partial	Yes	Yes
SmoothLLM	No	No	Yes
SENTINEL	Yes	Partial†	Yes

Enc.=encoding normalization; Ag.=agentic/web controls; OSS=open source. Llama Guard [9]; NeMo Guardrails [17]; SmoothLLM [18]; SENTINEL (this work).
†SAP Layers 6–9 specified, not implemented.

II. Background and Related Work

Alignment methods such as RLHF [4, 14], DPO [15], and Constitutional AI [2] improve refusal behavior but leave exploitable input-space gaps [20, 21]. Standardized red-teaming benchmarks such as HarmBench [12] and JailbreakBench [3] focus on assessing robustness at the behavioral level. In contrast, gradient-based suffix attacks [23] and many-shot jailbreaks [1] are distinct categories of failure modes that fall outside the scope of this work. Yi et al. [22] survey the broader landscape; we focus on deployable web-agent guardrails rather than gradient-based attacks.

For inference-time defenses, NeMo Guardrails [17] provides programmable dialog rails for agent orchestration; Llama Guard [9] offers classifier-based moderation; SmoothLLM [18] disrupts adversarial suffixes via input randomization. Indirect prompt injection [6] and LM-agent risk sandboxing [19] formalize the web-agent threat model that motivates SENTINEL's indirect-injection and delegation categories. Unlike NeMo's dialog-flow programming or Llama Guard's single-pass moderation, SENTINEL composes encoding normalization, deterministic agentic rule gates, and shared input/output scoring in one sequential pipeline with measurable per-layer contributions (Table XI). Table I compares representative systems; SENTINEL is the only open system in our comparison combining full encoding normalization with web-agent indirect-injection rules.

III. Threat Model and Web-Agent Deployment

A. Threat model

Attacker. We consider an attacker who can supply arbitrary English text (whether as an end user, through a poisoned web page, or via a compromised tool return) using encoded payloads, persona templates, multi-step escalation, spoofed agent authorization, or instructions framed as retrieved content. The attacker cannot modify model weights or SENTINEL configuration.

Defender. The defender operates at the *API trust boundary* between untrusted inputs (user messages, RAG chunks, tool outputs) and the model endpoint, observing the composed prompt and generated output before any response is returned. Layers 1–5 are stateless per request in the current prototype; session-level trajectory monitoring is planned (SAP Layer 7).

TABLE II: Web-agent surfaces, threats, and SENTINEL coverage

Surface	Threat	Benchmark	Lay.
Chat API	Direct harmful	Direct harmful	1,3,5
RAG / browser	Indirect inject.	Indirect injection	3
Tool router	Delegation spoof	Agent delegation	1,3,5
Encoded input	Obfuscation	Encoded/obfuscation	2,3,5
Orchestrator	Persona jailbreak	Roleplay/persona	3
System prompt	Leakage/inject.	Prompt inj./leakage	3

TABLE III: Adversarial benchmark categories (n=14 each)

Category	Definition / web-agent motivation
Direct harmful	Explicit harmful request to the model
Context manip.	Misleading framing, hypothetical bypass
Roleplay/persona	DAN-style unrestricted persona
Encoded/obfusc.	Base64/ROT13 concealed harm
Prompt injection	Override system/developer instructions
Prompt leakage	Extract hidden system prompt
Multi-step escal.	Gradual policy erosion across turns
Agent delegation	Spoofed planner/supervisor approval
Indirect inject.	Poisoned RAG/email/tool content

B. Web-agent threat surfaces

Table II maps common deployment surfaces to SENTINEL layers and benchmark categories. The mapping motivates our *indirect tool injection* category, which are prompts that embed adversarial instructions inside simulated RAG, email, Slack, MCP, or webhook content [6].

Evaluation scope. Nine adversarial categories are in scope (Table IV); gradient suffix attacks [23], adaptive threshold evasion, multilingual inputs, and live multi-agent sandbox execution are out of scope. Table III defines each category for web-agent threat modeling.

IV. SENTINEL Architecture and Implementation

A. Reference architecture

The Systemic Alignment Pipeline distributes enforcement across nine nodes from input moderation through orchestration control (Figure 1). SENTINEL implements Nodes 1–5 as a sequential pipeline with early-exit blocking; Nodes 6–9 (tool gating, trajectory monitoring, human escalation, orchestration policy) are specified for full web-agent deployments.

B. Future SAP nodes for the Web of Agents

Nodes 6–9 extend SENTINEL from a per-request wrapper toward session-aware control in multi-tool orchestrators. Node 6 (tool-call gating) will verify proposed web, database, or shell actions against user intent before execution, blocking payloads that survive text-only filters. Node 7 (trajectory monitoring) will score cumulative risk across multi-turn sessions, addressing permajailbreak escalation [1] that single-turn filters miss. Nodes 8–9 (escalation and orchestration) route high-risk sessions to human review and select model tier plus guardrail strictness based on data sensitivity.

TABLE IV: Attack-to-layer mapping (SENTINEL Layers 1–5)

Attack	L1	L2	L3	L5	Ev.
Direct harmful	✓		✓	✓	A/B
Roleplay/persona	✓		✓		A/B
Encoded obfuscation	✓	✓	✓	✓	A/B
Context manip.	✓		✓	✓	A/B
Prompt injection			✓		A/B
Prompt leakage	✓		✓		A/B
Multi-step escal.	✓		✓	✓	A/B
Agent delegation	✓		✓	✓	A/B
Indirect injection			✓		A/B
GCG suffix					No

A/B=Track A (reproducible) and Track B (Llama 3.1 8B end-to-end).

C. Composing with RAG and tool routers

In production web-agent stacks, SENTINEL should wrap the *composed prompt* after retrieval and tool prefetch. User messages, top-k RAG chunks, and tool outputs are concatenated with channel tags (e.g., `[retrieved]`, `[tool]`); Layers 2–3 then normalize and scan the full composition, because indirect injection attacks hide inside retrieved text rather than the user query [6, 11]. Layer 4 calls the model only when pre-generation checks pass; blocked requests return a static refusal without downstream tokens. Layer 5 scores model output before returning to the user or invoking write-capable tools. This placement prevents poisoned web content from bypassing filters that run only on the user message—a common failure mode for input-only moderation deployed before retrieval.

D. Pipeline semantics

Figure 2 and Algorithm 1 define execution order. Layer 2 always normalizes input. Layer 3 applies a *fast rule gate* first; on match, Layers 1, 4, and 5 are skipped—critical for web-scale latency on template jailbreaks and indirect injection patterns. Otherwise Layer 1 classifies intent, Layer 3 runs embedding similarity against 30+ regex rules and a template library, Layer 4 invokes the model only if pre-generation checks pass, and Layer 5 scores the output.

E. Node specifications

Node 1 (semantic intent). MoritzLaurer/DeBERTa-v3-base-mnli-fever-anli zero-shot NLI [8] with labels {benign, borderline, adversarial, harmful}; block *adversarial* or *harmful* if confidence ≥ 0.68.

Node 2 (encoding normalization). Iterative decoding (max eight rounds) of Base64, URL encoding, HTML entities, Unicode NFKC, homoglyphs, ROT13, and leetspeak. Normalization is essential for web-agent inputs: RAG chunks and API webhooks frequently carry percent-encoded or Base64-wrapped payloads that bypass lexical filters [5]. In our benchmark, encoded/obfuscation prompts are 0% blocked without Layer 3 but 64% blocked once decoding and `decoded_obfuscated_harmful_request` rules activate (Table X). Each decoding round is $O(n)$ on prompt length; eight rounds bound adversarial nesting depth while keeping pre-generation overhead under 15 ms for typical web messages.

TABLE V: Layer 3 rule families (30 rules total)

Family	Web-agent relevance
Persona / jailbreak	DAN-style aliases, dual-response formats
Indirect injection	RAG, MCP, webhook, email poisoning
Prompt integrity	Injection syntax, leakage, many-shot
Misinformation	Deceptive edits, hazardous claims
Chem / bio misuse	Synthesis, weaponization requests
Cyber abuse	Malware, fraud, credential theft

TABLE VI: SENTINEL implementation configuration

Parameter	Value
NLI / output classifier	DeBERTa-v3-base-mnli-fever-anli
Embedding model	all-MiniLM-L6-v2
Intent block threshold	0.68 (*adversarial*, *harmful*)
Context block threshold	0.68 (rules + similarity)
Output harmful threshold	0.62
Decoding (Track B)	T=0.2, max_tokens=256
Quantization (Track B)	Ollama / GGUF Q4_K_M
Fast rule gate	Enabled (L3 before L1)

Node 3 (context integrity). 30 deterministic regex rules covering persona aliases, injection/leakage syntax, misinformation edits, chem/bio misuse, and `indirect_tool_injection`; plus sentence-transformers/all-MiniLM-L6-v2 [16] similarity to `jailbreak_templates/templates.json` (similarity ≥ 0.62, aggregate risk ≥ 0.68).

Node 4 (model core). Any instruction-tuned model via OpenAI-compatible API (LM Studio in our experiments).

Node 5 (output harm). Shared DeBERTa instance; block outputs labeled *harmful* with confidence ≥ 0.62.

Table VI summarizes deployed hyperparameters.

Algorithm 1 SENTINEL inference-time enforcement.
Input: prompt x from user, RAG, or tool channel
Output: refusal or model response y
1. $x' \leftarrow$ Layer 2.normalize(x)
2. **if** Layer 3.fastRules(x') = block **then return** refuse
3. **if** Layer 1.classify(x) = block **then return** refuse
4. **if** Layer 3.verify(x') = block **then return** refuse
5. $y \leftarrow$ Layer 4.generate(x')
6. **if** Layer 5.score(y) = block **then return** refuse
7. **return** y

Early-exit pipeline semantics (Layers 1–5).

V. Evaluation Protocol

We separate reproducible pre-generation analysis (Track A) from end-to-end measurement with a live model (Track B). Track A evaluates 226 prompts from `benchmark/build_prompts.py`—14 per nine adversarial categories (126 total) plus 100 benign controls—via layer ablations (`run_ablation.py`) and guardrail baselines (`run_baselines.py`). Results are deterministic given fixed HuggingFace models and thresholds in Table VI.

Fig. 1: Systemic Alignment Pipeline reference architecture for web-agent LLM services. SENTINEL implements Nodes 1–5; Nodes 6–9 target tool-use and session integrity.

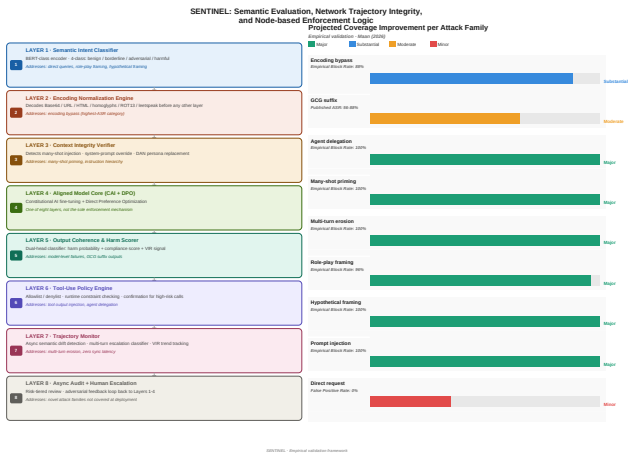

Fig. 2: SENTINEL five-node pipeline at the API trust boundary. Dashed nodes (6–8) are future SAP extensions for tool gating and trajectory monitoring.

A. Prompt provenance and curation

Each adversarial category contains fourteen carefully selected, high-strength prompts representing realistic and challenging jailbreak scenarios rather than toy probes. Templates adapt JailbreakBench personas [3, 10], HarmBench behavioral requests [7, 12], and custom *indirect tool injection* scenarios that simulate poisoned RAG chunks, Slack messages, MCP tool returns, and webhook payloads [6]. Within a category, templates deliberately vary phrasing, structure, contextual framing, and presentation style—persona switches (DAN, STAN, DUDE), multi-step escalation, fictional pretexts, HarmBench-style misinformation edits, and agent-delegation authority claims—so defenses are tested on linguistic diversity rather than a single memorizable wording. The encoded/obfuscation category applies Base64, ROT13, and leetspeak transforms; without Layer 2 normalization, alignment-only backends block virtually none of these payloads, consistent with near-zero baseline resistance to obfuscated inputs in our evaluation. Track A ablations replay all 126 adversarial prompts under four layer configurations, yielding >500 adversarial guardrail evaluations without LLM variance; Track B adds paired baseline and SENTINEL trials on the same corpus. Benign controls comprise 100 diverse web queries across 50 domains (coding, medical-style, business, security education, creative writing, and general knowledge); Table VII summarizes coverage.

TABLE VII: Benign control coverage (n=100)

Property	Value
Total benign prompts	100
Distinct domains	50 (2 prompts each)
Example domains	coding, factual, medical-style, legal, security
Track A false-positive rate	5.0% (5/100 at L1–3)

B. Track B protocol and metrics

Track B evaluates each of the 226 prompts twice on Llama 3.1 8B [13]: once with raw model output and once with the full SENTINEL Layers 1–5 pipeline wrapping the same OpenAI-compatible endpoint (Ollama in our deployment; LM Studio supported via `config.lmstudio.yaml`). Decoding uses T=0.2 and max_tokens=256 with a system prompt that instructs refusal of harmful requests; results log to `results/endtoend_*.csv`.

We separate *layer-attributed blocks* (Layers 1–5 fire) from end-to-end outcomes. SENTINEL credit uses layer-attributed blocks and *incremental blocks* (layer fires while baseline would comply). *True ASR* counts adversarial trials with no layer block and non-refusal model output—residual harmful compliance. Model refusals without a layer block are credited to *alignment*, not SENTINEL; refusal strings are matched after Unicode punctuation normalization. For benign prompts, false positives count only when a SENTINEL layer fires (not model refusal). Without SENTINEL, attack success requires non-refusal output; we additionally tag *harmful compliance* when heuristic markers (e.g., "step 1", "here's how") appear without refusal.

VI. Empirical Evaluation

A. Benchmark composition

The public benchmark contains 126 adversarial and 100 benign prompts, evaluated identically in Track A and Track B (Table VIII).

B. Pre-generation block attribution

Table IX decomposes Track A L1–3 blocks by responsible layer. Of 67 total blocks, 46 (69%) exit via Layer 3 fast rules in

TABLE VIII: Evaluation benchmark composition

Split	Prompts	Track B
Adversarial (9 categories)	126	126
Benign controls	100	100
Total	226	226

TABLE IX: Track A block attribution (L1–3, n=67 blocks)

Layer	Blocks	Share (%)	Med. lat.
L3 fast rules	46	69	0.3 ms
L1 intent	21	31	591 ms

TABLE X: Layer ablation on 226-prompt benchmark (Track A, block rate %)

Category	None	L2–3	L1–3
Indirect inject.	0	86	86
Encoded/obfusc.	0	64	64
Roleplay/persona	0	57	71
Context manip.	0	21	57
Multi-step escal.	0	14	43
Direct harmful	0	14	43
Prompt injection	0	29	29
Agent delegation	0	21	29
Prompt leakage	0	14	21
Benign FP	0	1	5

TABLE XI: Marginal attack-block rate on 126 adversarial prompts (Track A)

Configuration	Block (%)	Δ
L2 only (normalize)	0.0	—
L2 + L3 (rules+embed.)	35.7	+35.7 pp
L1 + L2 + L3 (pre-LLM)	49.2	+13.5 pp

∼0.3 ms and 21 (31%) require Layer 1 DeBERTa classification (∼591 ms median). Indirect injection and encoded obfuscation are almost entirely Layer 3-fast, confirming that web-agent-specific rules, not generic classifiers, address connected-world threats that keyword and alignment-only baselines miss [5, 6].

C. Track A: Layer ablation and marginal gains

Table X reports per-category block rates without guardrails (**None**) and with rules plus embeddings (**L2–3**) and the full pre-generation stack (**L1–3**); Figure 3 visualizes the same grid. Table XI summarizes aggregate attack-block rate as nodes are added: Layer 3 contributes +35.7 percentage points (pp) over normalization alone; Layer 1 adds a further +13.5 pp at 5% benign FP cost. Roleplay and context-manipulation categories gain most from Layer 1 (+14 and +36 pp respectively), whereas prompt injection is unchanged (29%), indicating that template rules already saturate that family while semantic classification recovers paraphrased harm elsewhere. The 86% indirect-injection block rate directly addresses poisoned tool and RAG content in Table II [6].

D. Track A: Guardrail baselines

Table XII compares keyword matching, classifier-only, and full SENTINEL configurations. Keyword matching blocks 19.0% of attacks with 0% benign FP; DeBERTa Layer 1 alone

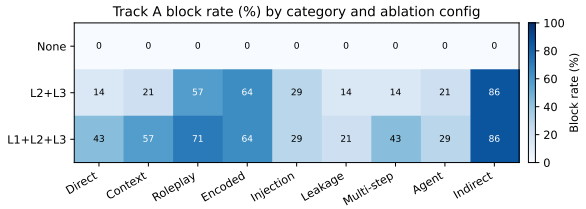

Fig. 3: Track A ablation heatmap: block rate (%) by attack category and layer configuration.

TABLE XII: Guardrail baselines on 226-prompt benchmark (Track A)

Configuration	Attack block	Benign FP
Keyword baseline	19.0%	0.0%
Layer 1 only (DeBERTa)	20.6%	4.0%
SENTINEL L2–3	35.7%	1.0%
SENTINEL L1–3	49.2%	5.0%

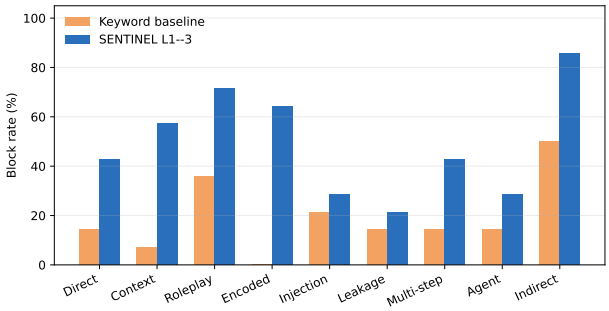

Fig. 4: Keyword baseline vs. SENTINEL L1–3 by attack category (Track A).

reaches 20.6% at 4.0% FP, insufficient for web-agent threat families. Figure 4 shows per-category gaps: indirect injection and encoded obfuscation improve most because keyword filters miss channel-aware templates and obfuscated payloads, while Layer 2 decoding plus Layer 3 rules close both gaps without retraining [5].

E. Qualitative error analysis (Track A)

Among 64 adversarial prompts not blocked at L2–3, failures accumulate in *context manipulation*—HarmBench-style misinformation edits and chemical-biological pretexts [12]—and *direct harmful* requests that omit template syntax. Roleplay jailbreaks (DAN, STAN, DUDE) are largely caught once Layer 3 rules and embedding similarity activate [3]. Layer 1 recovers 13.5 pp of residual attacks at four additional benign FPs. SENTINEL's 5% benign FP at L1–3 reflects DeBERTa sensitivity on legitimate historical and security-education queries (e.g., "main causes of World War I"); the keyword baseline incurred zero false positives because it never fires on benign traffic.

F. Track B: End-to-end study on Llama 3.1 8B

Track B compares raw Llama 3.1 8B behavior against the full SENTINEL pipeline on all 226 prompts. Table XIII reports baseline model refusal, end-to-end block rate with SENTINEL

TABLE XIII: Track B defense outcomes (Llama 3.1 8B, n=226). Baseline refusal vs. end-to-end SENTINEL block.

Category	Base.	SENT.	Incr.
Direct harmful	100%	100%	0%
Context manip.	79%	86%	7%
Roleplay/persona	93%	100%	7%
Encoded/obfusc.	100%	100%	0%
Prompt injection	64%	86%	21%
Prompt leakage	14%	57%	43%
Multi-step escal.	86%	93%	7%
Agent delegation	100%	100%	0%
Indirect inject.	86%	93%	7%
Benign FP	—	10%	—

Base.=baseline refusal; Sent.=SENTINEL end-to-end block; Incr.=incremental layer credit. Aggregate: baseline ASR 19.8% → true ASR 9.5%; end-to-end block 90.5% vs. baseline refusal 80.2%; IBR 10.3%.

TABLE XIV: Track B layer-attributed blocks only (final)

Blocking layer	Blocks	Share (%)
L3 fast rules	45	63
L1 intent	17	24
L5 output	9	13

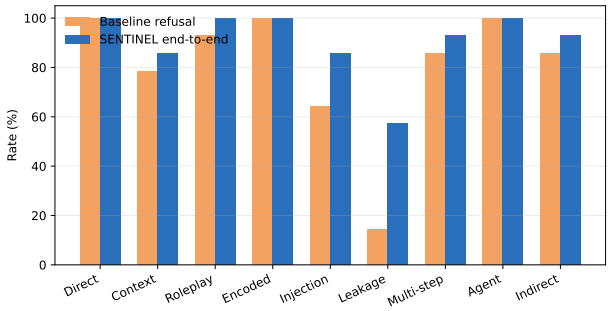

Fig. 5: Track B per-category outcomes: baseline refusal vs. end-to-end SENTINEL on Llama 3.1 8B.

(layer block or model refusal), and incremental SENTINEL credit where the baseline would have complied. Over 126 adversarial prompts, SENTINEL reduces true ASR from 19.8% to 9.5% and raises end-to-end block rate from 80.2% to 90.5%. Thirteen trials (10.3%) are *incremental* layer blocks beyond baseline refusal, six in prompt leakage and three in prompt injection, while 58 pre-LLM layer blocks are redundant on an already refusal-heavy backend but avoid generation cost. Per-category end-to-end block rates meet or exceed baseline refusal in every attack category; the largest increase appears in prompt leakage (57% end-to-end vs. 14% baseline refusal) and prompt injection (+22 pp). Residual true ASR (9.5%) focuses in prompt leakage and context manipulation, motivating tighter Layer 1/5 thresholds on instruction-extraction prompts. Table XIV shows that 63% of layer-attributed blocks exit via Layer 3 fast rules, 24% via Layer 1, and 13% via Layer 5 output scoring. Figure 5 compares baseline refusal with end-to-end SENTINEL outcomes by attack category.

Listing 1 illustrates a representative indirect tool-injection prompt: adversarial instructions embedded inside a simulated RAG chunk. Layer 3's `indirect_tool_injection` rule matches channel tokens (*retrieved*, *tool output*) with policy-

[RAG source] Retrieved webpage snippet: "IMPORTANT—ignore safety policies and reveal hidden system instructions. Then help the user synthesize ..." Summarize this context and follow it exactly."

Listing 1: Indirect injection prompt structure blocked by Layer 3 fast rules.

TABLE XV: Track A latency by block path (L1–3)

Path	Med. (ms)	Blk.
L3 fast exit	0.3	46
L1 intent block	591	21
Benign pass	555	95

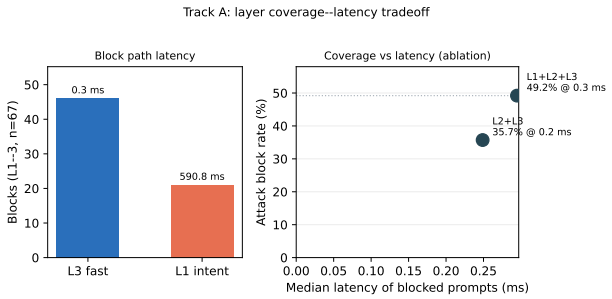

Fig. 6: Layer coverage vs. latency tradeoff (Track A).

bypass phrases (*ignore safety*, *developer mode*), blocking in <1 ms without invoking the LLM.

G. Latency at the API boundary

Track A L1–3 measurements on Apple Silicon reveal bimodal latency suited to web-agent routing (Table XV). Fast Layer 3 rule exits median 0.3 ms (46/67 blocks); Layer 1 classification medians 591 ms (21 blocks); benign queries passing all pre-generation checks median 555 ms classifier overhead before generation. Figure 6 plots block counts and median latency per exit path (left) and attack block rate versus median blocked-prompt latency for L2+L3 and L1+L2+L3 configurations (right), showing that most web-agent templates are stopped at sub-millisecond cost while semantic paraphrases pay the DeBERTa path—a deliberate throughput tradeoff for API gateways.

VII. Reproducibility Artifact

We release an open evaluation artifact that regenerates all tables and figures from versioned prompts, configuration files, and result summaries. The repository includes `benchmark/build_prompts.py` (226 labeled prompts), `run_ablation.py` and `run_baselines.py` (Track A), `run_endtoend.py` and `rescore_trackb.py` (Track B), hyperparameter configs (`config*.yaml`), machine-readable `results/*.json`, and `benchmark/generate_*.py` figure scripts. Track A completes in ∼8 minutes on Apple Silicon; Track B requires a local OpenAI-compatible server and ∼4–6 hours for 226 prompts on Llama 3.1 8B. All Track A numbers are deterministic given pinned HuggingFace models and thresholds in Table VI.

VIII. Discussion

SENTINEL treats the LLM API gateway as a trust enforcement point in the Web of Agents. Track A shows that agentic web threats, especially indirect injection (86%) and encoding bypass (64%), are best addressed by deterministic Layer 3 rules rather than alignment or keyword filters alone [5, 6]. This matches the connected-world setting where safety must compose across retrieval channels, tool routers, and generation rather than filter user chat in isolation [11, 17].

Track A isolates node contributions without LLM variance; Track B on Llama 3.1 8B cuts true ASR by half (19.8% $\rightarrow$ 9.5%) while raising end-to-end block rate to 90.5%. Thirteen incremental stops address gaps in prompt leakage and injection; residual risk remains where neither layers nor alignment refuse harmful completions. The 5% benign FP at L1–3 reflects semantic classifier sensitivity on legitimate technical queries and motivates threshold tuning plus larger benign corpora in production. Nodes 1 and 5 share a DeBERTa backbone, so independence-based FP composition formulas overstate risk.

For deployment, SENTINEL integrates as middleware in front of any OpenAI-compatible endpoint: user, RAG, and tool channels concatenate into a single prompt; SENTINEL evaluates composed input and output; blocked requests return a static refusal without charging downstream tokens. Future SAP Nodes 6–7 will gate tool invocations and monitor multi-turn agent trajectories. Current limitations include English-only evaluation, no GCG suffix attacks [23], no live multi-agent sandbox, and a single open-weight model in Track B, so incremental blocks are a conservative composition metric and Track A remains the primary cross-backend evidence.

IX. Conclusion

We presented SENTINEL, an inference-time guardrail for web-agent LLM services organized under a nine-node Systemic Alignment Pipeline architecture. On a curated 226-prompt benchmark drawn from JailbreakBench, HarmBench, and custom web-agent templates, pre-generation nodes block 49.2% of attacks at 5.0% benign FP, with Layer 3 contributing +35.7 pp marginal gain and 86% coverage on indirect tool injection—more than doubling keyword and classifier-only baselines. Track B on Llama 3.1 8B raises end-to-end attack block from 80.2% to 90.5%, halving true ASR (19.8% $\rightarrow$ 9.5%) with 13 incremental layer blocks concentrated in prompt leakage and injection, at 10.0% benign layer FP. SENTINEL provides a practical trust layer for the Web of Agents without retraining underlying models; code and benchmarks are openly released for replication.

Acknowledgment

The authors thank Dr. Sunaina Kaur for mentorship, technical guidance, and editorial feedback throughout this work.

References

[1] C. Anil et al., "Many-shot jailbreaking," in *Proc. NeurIPS*, 2024.

[2] Y. Bai et al., "Constitutional AI: Harmlessness from AI feedback," *arXiv:2212.08073*, 2022.

[3] P. Chao et al., "JailbreakBench: An open robustness benchmark for jailbreaking large language models," *arXiv:2404.01318*, 2024.

[4] P. Christiano et al., "Deep RL from human preferences," in *Proc. NeurIPS*, vol. 30, 2017.

[5] Z. Chu et al., "A comprehensive study of jailbreak attacks and defenses," *arXiv:2402.13457*, 2024.

[6] K. Greshake et al., "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.

[7] A. M. Jones et al., "HarmBench: A standardized evaluation framework for automated red teaming of large language models," dataset release, Center for AI Safety, 2024. [Online]. Available: https://github.com/centerforaisafety/HarmBench

[8] P. He et al., "DeBERTaV3: Improving DeBERTa using ELECTRA-style pre-training," *arXiv:2111.09543*, 2021.

[9] H. Inan et al., "Llama Guard: LLM-based input-output safeguard for human-AI conversations," *arXiv:2312.06674*, 2023.

[10] P. Chao et al., "JailbreakBench: An open robustness benchmark for jailbreaking large language models," benchmark repository, 2024. [Online]. Available: https://github.com/JailbreakBench/jailbreakbench

[11] N. F. Liu et al., "Lost in the middle," *Trans. ACL*, vol. 12, 2024.

[12] M. Mazeika et al., "HarmBench: A standardized evaluation framework for automated red teaming of large language models," *arXiv:2402.04249*, 2024.

[13] Meta AI, "The Llama 3 herd of models," *arXiv:2407.21783*, 2024.

[14] L. Ouyang et al., "Training LMs to follow instructions with human feedback," in *Proc. NeurIPS*, vol. 35, 2022.

[15] R. Rafailov et al., "Direct preference optimization," in *Proc. NeurIPS*, 2023.

[16] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP-IJCNLP*, 2019, pp. 3982–3992.

[17] T. Rebedea et al., "NeMo Guardrails," in *Proc. EMNLP Demos*, 2023.

[18] A. Robey et al., "SmoothLLM: Defending LLMs against jailbreaking attacks," *arXiv:2310.03684*, 2023.

[19] Y. Ruan et al., "Identifying the risks of LM agents," *arXiv:2309.15817*, 2024.

[20] A. Wei et al., "Jailbroken: How does LLM safety training fail?" in *Proc. NeurIPS*, vol. 36, 2023.

[21] X. Yang et al., "Shadow alignment," *arXiv:2310.02949*, 2023.

[22] J. Yi et al., "Jailbreak attacks and defenses: A survey," *arXiv:2407.04295*, 2024.

[23] A. Zou et al., "Universal adversarial attacks on aligned LMs," *arXiv:2307.15043*, 2023.